Total Type Script

Ways to install TS:

1. Global : npm install -g typescript

   tsc -v ⇒ to get version

   tsc file.ts ⇒ to use TS

2. Locally : npm install -D typescript

   npx tsc --init : Initialize config

   npx tsc : Run compiler

3. Using npx directly (no need to install)

   npx tsc --init : Initialize config

   npx tsc file.ts : To run a file

4. using any package manager like pnpm/yarn etc

   pnpm add -D typescript ; bun init

5. Using TS with Node

   npm install ts-node typescript -save-dev

6. Using TS via Frameworks.

   npm create vite@latest

   your package.json should have :

```
{"scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
}
```

7. Using bun or deno

   deno -v : for version

   deno --init

   deno run main.ts : Run program

In bun,

to target a single file

target all files in src

```
"scripts": {
  "dev": "bun run ./src/index.ts", or  bun run src/*
  "build": "bun build ./src/index.ts --outdir ./dist"
}
```

2

or

src/*

Note: Always use a single entry point

Any

→ let name: any; → type = anything

Unknown → type safe version of any, means this value can be anything, but we must check before using it

let x: unknown;

Void (no return) eg: Returns no useful value.

```
function logMessage(): void {
    log("Hello");
}
```

Never : This fn never finishes normally, never returns at all

eg. fun throwError(): never {
    throw new Error("something broke");
}

Enum in JS (not preferred) : A way to define a set of named constants.

const enum TeamStrength {Weak = 1, Good, Strong};

let yellow = TeamStrength.Weak;

Log (yellow) ⇒ gives 1.

Tuple : A special array whose length and type of each position is fixed.

Tuple : A fixed length array where each element has a specific type.

```
let album : [string, number] = ["Mantra", 2025]
```

Here the order also matters.

Array in TS : An array is list of values of the same type. Here all elements are of same type and the size/length is not fixed.

```
let nums : number[] = [1,2,3];
```

Object : An object is a collection of key-value pairs with defined types.

eg:

```
let mentor : { id : number , name : string } = { id : 4 , name : xyz }
```

eg:

```
address? : { street : string , pin : number }
```

↑ becomes optional

Function in TS :

Functions have typed parameters + typed return value

```
function calculatePrice (team : ~~name~~ string) : number {
if (team.toLowerCase() === "xx") return 15e10;
}
```

↑ type of parameter

↑ return type

Type Annotation vs Type Inference

1. **Type Annotation** : We explicitly tell TS the type

```
let name : string = "Jaani";
```

2. **Type Inference** : We let TS guess the types automatically

```
let name = "Jaani";
```

Note : TS is smart enough to guess type, so we only write types when needed.

Type Alias : This lets you give a custom name to a type.

eg:

```
type Team = {
    name : string;
    getSquad : () => string[];
    price : number;
    isBanned : boolean;
}
```

Note : Though it looks like classes in JS, it is not the same.
Type Alias : only exists in TS, but have no methods inside it. You just have the structure (even of methods)

Alias only gives shape of data so is not same as classes in TS.

Union (Means the value can be of many types)

```
fn KgToLbs (weight : number | string) : number {
```

← return type

type can be both a number or a string

Intersection (&) → combines all properties into one type.

```
type Male = {
    speak : () => void;
};
type Lion = {
    roar : () => void;
};
type Narasimha = Male & Lion;
```

Null : Ts is strict, so it won't allow "null" unless we explicitly adds it.

```
let name : string | null = null;
```
(only string or null is allowed)

If you give : let hello : string | null | undefined;

so hello = null;
hello = undefined;
hello = "hello";
} all these are allowed

Nullish Coalescing Operator (??) → used to give a default value only when value is null or undefined.

For example, if this value is missing, use a fallback value.

```
eg. let speed : number | null | undefined = null;
    let ride = {
        speed : speed !== null ? speed : 49,
    };
```

log(ride.speed) gives 49

If speed is not null then use speed's value given

For ??

```
let ride = {
    speed : speed ??49,
};
```

If speed is null or undefined, use 49 otherwise use speed

Type Assertions In TS

Here we do not change the actual data/input but we tell it manually the type.

```
let phone = document.getElementById("phone") as
            HTMLInputElement | null;

let phoneNo = phone?.value;
log(phoneNo);
```

Re-Export : Re-exporting is used to forward exports. It is a clean way to pass exports through files, so you don't have to import it everywhere.

~~ex~~ without re-export :

```
import {add} from "./utils/math";
import {sub} from "./utils/math",
```

With re-export :

```
export * from "./math";
```

{yyi, will only work for named exports, not default ones}

Interface VS Type

Both are used to define shape of data, but Interface is used when working with objects which needs to be extended. ~~Type give~~

Generics in TS (general type)

```
fun getResponse(data:any) {
    return {
        success: true,
        status : 200,
        data,
    };
}
```

[without generics]

⇒ with generics

```
~~fun getResponse(data~~:
fun getResponse<T>(data:T[]) {
    return {
        success: true,
        Status :200,
        data,
    );
}
const num = getResponse([1,2,3]).data
const strings = getResponse(["hello", "I", "am"]).data;
```

Here T is like a placeholder

Generics allow us to write reusable code with type safety. It ensures that our input & Output types remain consistent without us using any (any creates ambiguity)

Type vs interface continued:

Type gives us more flexibility like unions, Intersections etc.

Generics interfaces in TS → works like a reusable template where we don't fix the type beforehand.

Follow:
X: jaaaani404

Assignment : Solve questions on TS Hero.